

SKILL WEEK -3

Task 1:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0)
        {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd OSSP
[(base) saiabburi@Geyasree-MacBook-Air OSSP % nano task1.c
[(base) saiabburi@Geyasree-MacBook-Air OSSP % gcc task1.c -o task1
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ./task1
myshell> Hello
You entered: Hello
myshell> ls
You entered: ls
myshell> exit
Exiting shell...
(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

Task 2 :

UW PICO 5.09

File: task2.c

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    ssize_t bytesRead, bytesWritten;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 == -1) {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 == -1) {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    while ((bytesRead = read(fd1, buffer, sizeof(buffer))) > 0) {

        bytesWritten = write(fd2, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }

    if (bytesRead == -1) {
        perror("Error reading File1.txt");
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
```

This is File1.
This content will be copied to File2.
Operating System System Calls.

```
((base) saiabburi@Geyasree-MacBook-Air OSSP % nano File1.txt
((base) saiabburi@Geyasree-MacBook-Air OSSP % nano File1.txt
((base) saiabburi@Geyasree-MacBook-Air OSSP % cat File1.txt
This is File1.
This content will be copied to File2.
Operating System System Calls.
((base) saiabburi@Geyasree-MacBook-Air OSSP % nano task2.c
((base) saiabburi@Geyasree-MacBook-Air OSSP % gcc task2.c -o task2
((base) saiabburi@Geyasree-MacBook-Air OSSP % ./task2
File copied successfully.
((base) saiabburi@Geyasree-MacBook-Air OSSP % cat File2.txt
This is File1.
This content will be copied to File2.
Operating System System Calls.
((base) saiabburi@Geyasree-MacBook-Air OSSP % ls
File1.txt          bin                fork_exec.c        main_loop.c        task2
File2.txt          command_executer.c include             obj                task2.c
LICENSE            command_executor   main                scripts            tests
Makefile           command_executor.c main.c              src
README.md          docs               main.c.save        task1
assets             fork_exec          main_loop           task1.c
(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

Task 3:

```
((base) saiabburi@Geyasree-MacBook-Air OSSP % nano task3.c
((base) saiabburi@Geyasree-MacBook-Air OSSP % gcc task3.c -o task3
((base) saiabburi@Geyasree-MacBook-Air OSSP % ./task3
myshell> Hello
Hello
You entered: Hello
myshell> ./task3
./task3
You entered: ./task3
myshell> helo
helo
You entered: helo
myshell> o
o
You entered: o
myshell> exit
exit
You entered: exit
Exiting shell...
(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

```
#include <stdio.h>
#include <unistd.h>

#define MAX 100

int main() {
    char buffer[MAX];
    int index = 0;
    char ch;

    printf("myshell> ");
    fflush(stdout);

    while (1) {
        if (read(STDIN_FILENO, &ch, 1) <= 0) {
            break;
        }

        if (ch == '\n') {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            if (index == 4 &&
                buffer[0] == 'e' &&
                buffer[1] == 'x' &&
                buffer[2] == 'i' &&
                buffer[3] == 't') {
                printf("Exiting shell...\n");
                break;
            }

            index = 0;

            printf("myshell> ");
            fflush(stdout);
        }

        else if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }

        else {
            if (index < MAX - 1) {
                buffer[index++] = ch;
                write(STDOUT_FILENO, &ch, 1);
            }
        }
    }

    return 0;
}
```

Task 4 :

UW PICO 5.09

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_INPUT 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;

void print_prompt() {
    printf("myshell> ");
    fflush(stdout);
}

void save_history(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void clear_line(int length) {
    for (int i = 0; i < length; i++) {
        printf("\b \b");
    }
}

int main() {
    struct termios oldt, newt;
    char buffer[MAX_INPUT];
    char ch;
    int index;
    int history_index;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {
        index = 0;
        history_index = history_count;
    }
}
```

```

memset(buffer, 0, sizeof(buffer));

print_prompt();

while (1) {
    read(STDIN_FILENO, &ch, 1);

    if (ch == '\n') {
        buffer[index] = '\0';
        printf("\n");

        if (strcmp(buffer, "exit") == 0) {
            tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
            printf("Exiting shell...\n");
            return 0;
        }

        printf("You entered: %s\n", buffer);

        save_history(buffer);
        break;
    }

    if (ch == 127) {
        if (index > 0) {
            index--;
            buffer[index] = '\0';
            printf("\b \b");
            fflush(stdout);
        }
        continue;
    }

    if (ch == 27) {
        char seq[2];

        read(STDIN_FILENO, &seq[0], 1);
        read(STDIN_FILENO, &seq[1], 1);

        if (seq[0] == '[' && seq[1] == 'A') {
            if (history_count > 0 && history_index > 0) {
                clear_line(index);

                history_index--;
                strcpy(buffer, history[history_index]);

                index = strlen(buffer);

                printf("%s", buffer);
            }
        }
    }
}

```

```

        if (seq[0] == '[' && seq[1] == 'A') {
            if (history_count > 0 && history_index > 0) {
                clear_line(index);

                history_index--;
                strcpy(buffer, history[history_index]);

                index = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        else if (seq[0] == '[' && seq[1] == 'B') {
            if (history_index < history_count - 1) {
                clear_line(index);

                history_index++;
                strcpy(buffer, history[history_index]);

                index = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        continue;
    }

    if (index < MAX_INPUT - 1) {
        buffer[index++] = ch;
        printf("%c", ch);
        fflush(stdout);
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```

```

(base) saiabburi@Geyasree-MacBook-Air OSSP % nano task4.c
(base) saiabburi@Geyasree-MacBook-Air OSSP % gcc task4.c -o task4
(base) saiabburi@Geyasree-MacBook-Air OSSP % ./task4
myshell> Hello
You entered: Hello
myshell> linux
You entered: linux
myshell> test
You entered: test
myshell> helloo worldd
You entered: helloo worldd
myshell> exit
Exiting shell...
(base) saiabburi@Geyasree-MacBook-Air OSSP %

```

Task 5 :

UW PICO 5.09

File: task5.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

struct Node {
    char *command;
    struct Node *next;
};

int main() {
    char *buffer;
    int size = INITIAL_SIZE;
    int length = 0;
    int ch;

    struct Node *head = NULL;
    struct Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter commands.\n");
    printf("Type exit to stop.\n");

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        length = 0;

        while ((ch = getchar()) != '\n' && ch != EOF) {

            if (length >= size - 1) {
                size *= 2;

                char *temp = realloc(buffer, size);

                if (temp == NULL) {
                    printf("Memory reallocation failed.\n");
                    free(buffer);
                    return 1;
                }

                buffer = temp;
            }
        }
    }
}
```

```

        buffer[length++] = ch;
    }

    buffer[length] = '\0';

    if (strcmp(buffer, "exit") == 0) {
        break;
    }

    if (length == 0) {
        continue;
    }

    struct Node *newNode = malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        break;
    }

    newNode->command = malloc(strlen(buffer) + 1);

    if (newNode->command == NULL) {
        free(newNode);
        printf("Memory allocation failed.\n");
        break;
    }

    strcpy(newNode->command, buffer);
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }

    printf("Stored command: %s\n", buffer);
}

printf("\nCommand History:\n");

struct Node *current = head;

while (current != NULL) {
    printf("%s\n", current->command);
    current = current->next;
}

current = head;

while (current != NULL) {
    struct Node *temp = current;
    current = current->next;

    free(temp->command);
    free(temp);
}

free(buffer);

printf("Memory released successfully.\n");

return 0;
}

```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % nano task5.c
[(base) saiabburi@Geyasree-MacBook-Air OSSP % gcc task5.c -o task5
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ./task5
```

Enter commands.

Type exit to stop.

myshell> Hello

Stored command: Hello

myshell> Operating Systems

Stored command: Operating Systems

myshell> exit

Command History:

Hello

Operating Systems

Memory released successfully.

```
(base) saiabburi@Geyasree-MacBook-Air OSSP % ls
```

File1.txt	README.md	command_executor	fork_exec.c	main.c.save	scripts	task2	task4	tests
File2.txt	assets	command_executor.c	include	main_loop	src	task2.c	task4.c	
LICENSE	bin	docs	main	main_loop.c	task1	task3	task5	
Makefile	command_executor.c	fork_exec	main.c	obj	task1.c	task3.c	task5.c	

```
(base) saiabburi@Geyasree-MacBook-Air OSSP %
```